Evaluating Packet Processing Frameworks

Henrique B. Brum

1 Introduction

The constant increase in link and NIC bandwidth [1, 2] puts huge pressure on the packet processing frameworks of the operating system (OS). They must handle a massive amount of packets from the NIC while efficiently delivering all network traffic to the destination application. To meet these demands, numerous tools exist, ranging from traditional kernel-space mechanisms to modern kernel-bypass frameworks, each offering distinct advantages and trade-offs. Some of the most well-known and widely used frameworks[1] include AF_PACKET [8], AF_XDP [3, 16], DPDK [13] and PF_RING [10].

To understand which tools provide the best performance, various works have analyzed and evaluated them in different scenarios. Some works focus on specific strategies, especially AF_XDP and the tuning techniques that can increase its performance [9, 12], while others provide side-by-side comparisons using simple baseline applications [6, 7], such as packet drop or forwarding tasks. Only a few works have expanded their scope to include more complex applications like User Plane Functions (UPF) from the 5G ecosystem [17] or Network Intrusion Detection Systems (NIDS) [15].

In this paper, we evaluate and compare four widely used packet processing technologies—AF_PACKET, AF_XDP, DPDK, PF_RING—across different operational scenarios. Unlike existing work, we evaluate all frameworks both in a simple scenario using a minimal packet-counting application and in a complex scenario where they are integrated into Suricata [11]. Performing both evaluations within the same study provides a direct comparison between raw packet processing capabilities and application-level integration efficiency. Moreover, we evaluate them under diverse workloads, encompassing not only multiple link speeds and packet sizes, but also three distinct flow profiles with varying flow length distributions. This analysis is especially important for understanding how such systems scale when using multiple threads and Receive Side Scaling (RSS). The source code and results are available https://github.com/HenriqueBBrum/Evaluating-Packet-Processing-Frameworks.

Our findings indicate that while DPDK delivers unmatched raw processing capability, achieving complete packet visibility at 90Gbps with a single thread in two out of three scenarios, this raw performance does not automatically translate to superior real-world application throughput. When evaluated within Suricata, AF_XDP demonstrated the best balance between high-speed packet capture efficiency and overall application performance, leading Suricata to inspect more

[1] While not all of these solutions are usually classified/referred as "frameworks", we adopt this term in this paper for simplicity.

than 90% of all transmitted packets (given sufficient threads), whereas the other frameworks failed to inspect more than 45% of them. These results highlight how selecting the right packet processing framework depends not only on its raw packet processing performance, but also on the complexities of working with it and the internal structures it provides to support more complex packet and flow processing.

2 Background

This section presents the background concepts and technologies that the evaluated packet processing frameworks depend on.

2.1 NAPI

Interrupting the a core for every single incoming packet would greatly reduce the system's performance under high-throughput traffic. To avoid this cost, NAPI was created to provide a more efficient event-handling mechanism. To start with, whenever a packet arrives and is transferred to the main memory via Direct Memory Access (DMA), the NIC triggers a hardware interrupt (IRQ) to the CPU, and NAPI begins running. Instead of executing the full packet processing directly within this IRQ context, the driver disables further hardware interrupts and schedules a `NET_RX_SOFTIRQ` (software interrupt), which calls a polling function to read incoming packets from the NIC's receive (RX) ring. Delegating processing to the *softirq* context ensures the CPU is not entirely blocked by the hardware interrupt. It continues polling packets until it reaches a budget (300 packets by default) or the polling cycle is over. Finally, the packets are passed to upper-layer processing, and hardware interrupts are re-enabled.

2.2 RSS

The existence of processing cores and threads within a single CPU allows for a much greater degree of parallelism. Instead of handling all incoming packets on the same core, multi-core CPUs enable each core to process its own subset of packets in parallel. RSS determines, in hardware, to which core each packet should be distributed to and processed. It uses a hashing algorithm (usually Toeplitz) on specific fields of the packet header to map packet processing to an specific core. Typically, the five-tuple (or six-tuple when including the VLAN tag) is hashed to ensure that a single core handles an entire flow, avoiding the need to exchange packets between cores during flow processing. RSS happens before NAPI, linking each RX queue/ring to its own NAPI instance.

2.3 XDP

The eXpress Data Path (XDP) [6] provides the earliest inspection point in the kernel for packet processing. It executes right after the NIC writes the packet

into memory via DMA, but before the `sk_buff` structure is allocated for the raw packets. This privileged location allows XDP programs to operate with minimal overhead, making it the ideal spot for filtering or redirecting packets.

3 Packet Capture Tools

The four selected frameworks represent established mechanisms within the Linux ecosystem (AF_PACKET, AF_XDP, DPDK), as well as industry-made alternatives (PF_RING). Since our experiments focus exclusively on the RX path, we focus on explaining this path for each of the strategies.

3.1 AF_PACKET

AF_PACKET [8] is the standard Linux socket interface for capturing raw packets. It has been continuously updated over the years, with the latest version, `TPACKET_V3`, incorporating *mmap* for zero-copy memory sharing between the kernel space and user space. Figure 1 presents a diagram of the latest AF_PACKET architecture.

Once a packet arrives at the NIC, it is copied to the main memory through DMA to an RX ring buffer. Through the NAPI interface, packets are processed in batches: an *sk_buff* structure is allocated for each packet, and the packet is cloned. The original packet continues up the network stack to be processed by different kernel modules tools, such as *nftables*, while the cloned packet is delivered to the AF_PACKET socket for processing.

To scale across multiple threads, AF_PACKET provides a socket fanout options (`PACKET_FANOUT`) that can work in collaboration with RSS or not. Various load-balancing strategies exist to distribute traffic across sockets, such as `PACKET_FANOUT_HASH`, where all packets of a flow are directed to the same socket,

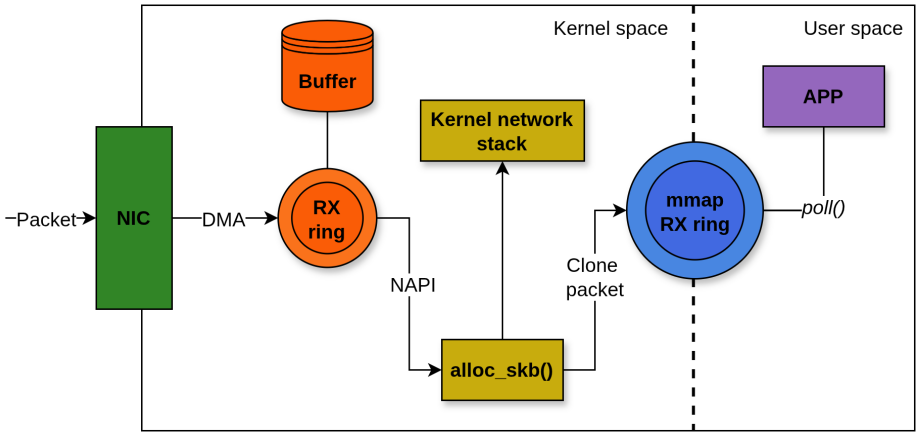


Fig. 1: AF_PACKET's packet RX path.

and `PACKET_FANOUT_CPU`, which considers the RSS decision and ensures that packets are processed on the core they arrived. In general, strategies that consider the decision of the RSS stage have better performance.

3.2 PF_RING

PF_RING [10] is an open-source packet capture framework developed by ntop. It exists in two main variants: vanilla and zero-copy (PF_RING ZC). The vanilla variant is free and operates similarly to the aforementioned AF_PACKET, though with less flexibility when handling multi-threading and integration with RSS. The zero-copy version is a proprietary/paid module that bypasses the kernel, allowing packets to pass directly to user-space applications. In this work, only vanilla PF_RING is evaluated.

3.3 AF_XDP

Like AF_PACKET, AF_XDP [3, 16] is a socket family in the Linux kernel. Differently, it leverages XDP to significantly boost its performance. AF_XDP consists of two main components: an XDP program that intercepts packets before socket buffer allocation (*sk_buff*) and forwards them directly to the AF_XDP socket; and the socket itself, which manages incoming and outgoing packets through a shared memory space called UMEM. Figure 2 shows how an incoming packet is processed by AF_XDP.

Both the kernel and user space share the UMEM area, where the actual packet data is stored. Four ring buffers handle the access to this shared space and track the state of each memory block. The two rings related to RX packets are:

– **Fill Ring**: The user-space application populates this ring with memory blocks (descriptors) for the kernel to write incoming packet data into.

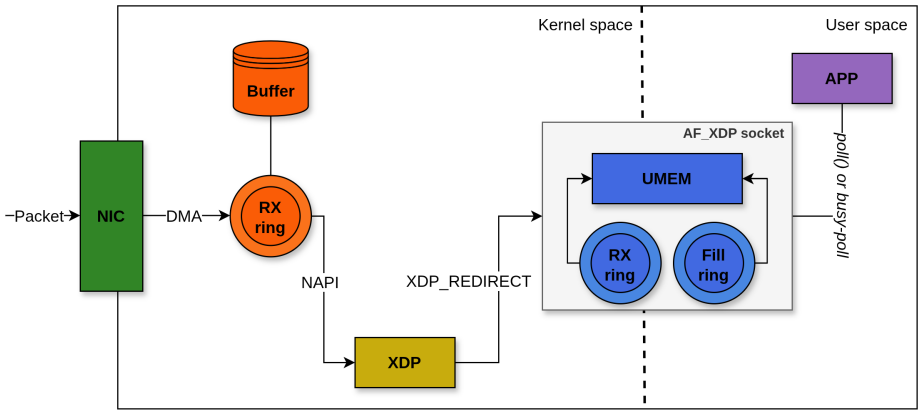


Fig. 2: AF_XDP's packet RX path.

– **RX Ring**: The kernel updates this ring once it writes RX packets into the allocated UMEM blocks, making them available for user-space consumption.

During packet reception, the user first allocates available blocks for the kernel by populating the Fill ring. The kernel consumes these descriptors, copies or writes incoming packet payload into the designated UMEM blocks, and updates the RX ring. The user application then reads from the RX ring, processes the newly arrived packets according to its internal logic, and frees the UMEM memory. The TX process follows an inverse workflow, where the user populates the TX ring with packets and the kernel consumes them for network transmission.

Beyond bypassing a substantial portion of the Linux network stack, AF_XDP supports a true zero-copy mode where the UMEM is directly accessible through DMA by the NIC, eliminating the overhead of copying packet data from a kernel-only buffer into the UMEM. Finally, when scaling across multiple threads with RSS, AF_XDP binds each processing socket to a specific hardware queue, aligning directly with the RSS mapping logic.

3.4 DPDK

Intel DPDK (Data Plane Development Kit) [13] is a packet processing framework that entirely bypasses the kernel, with all packet interactions managed by DPDK libraries in user space. For Intel NICs (and generic NICs), it unbinds the receiving port from the Linux kernel driver, completely hiding it from the OS. For Mellanox (NVIDIA) NICs, binding is unnecessary because the latest driver (*mlx5*) is bifurcated, meaning the interface remains visible and accessible to the Linux kernel. After the initial configuration stage (which requires kernel intervention) all subsequent network-related operations occur without kernel involvement.

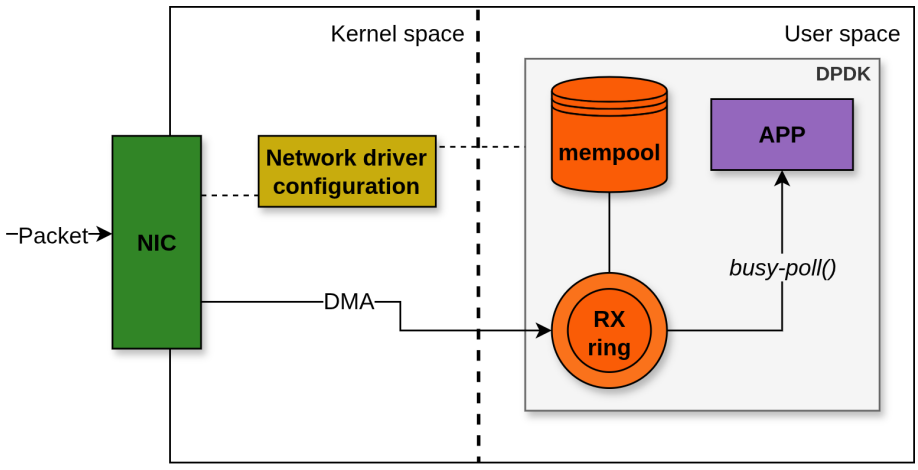


Fig. 3: DPDK's packet RX path.

In practice, DPDK packet processing functions similarly to the initial stages of the previously mentioned tools. The NIC uses DMA to write packets directly into the main memory, more specifically into pre-allocated *mempools* that leverage *hugepages* to maximize performance. Packet descriptors are maintained in a ring buffer containing pointers to the actual payload locations. The packets themselves are stored in an *rte_mbuf* structure (DPDK's simplified counterpart to *sk_buff*), making them immediately accessible for the application to process. Lastly, DPDK provides its own threading API, integrating directly with RSS by assigning worker threads to specific hardware threads.

4 Experiments Setup

This section presents the experiments setup and the format of each experiment type.

4.1 Setup

Our setup consists of one bare-metal server equipped with an AMD EPYC 9224 CPU (24 cores and 48 threads) that runs the desired packet capture frameworks and applications while a virtual machine (VM) with 16 cores is responsible for generating the network traffic with TRex[2]. Both machines are equipped with NVIDIA BlueField-2 DPUs and are directly connected via a 100Gbps link.

4.2 Experiments Format

There are three sets of experiments: (1) one single processing thread handling increasing throughput and different packet sizes to assess raw packet performance; (2) multiple threads with different flow profile to evaluate how each strategy scales; and finally, (3) Suricata experiments with different number of threads to understand how raw packet processing translates to actual application-level processing. For both the first and second experiment type, we created for AF_PACKET, AF_XDP and DPDK a basic program that simply counts the number of packet and bytes received, whereas for PF_RING, we used the *pfcount_multichannel* application provided by the developers themselves, which performs the same simple calculation. Lastly, we configured[3] the NIC driver identically across all strategies, setting the RX ring to 8192 entries (the maximum value) and employing the 5-tuple with the Toeplitz function for RSS.

Regarding the experiments traffic, in the single-thread scenario, we used TRex to generate one UDP flow at 1, 25, 40 and 90Gbps and with packets of 64, 512 and 1400 bytes for 20 seconds. Additionally, each tool was evaluated across different configurations in terms of buffer size. Differently, for the multi-threaded and Suricata experiments, we used TRex to generate flows following

[2] https://trex-tgn.cisco.com/

[3] For DPDK, we configured RSS within the application itself, whereas for the other tools, configuration was performed using *ethtool*.

different flow length distributions for 30 seconds. Figure 4 shows the distribution of the three evaluated sources: AGH [5], which contains campus network traffic; BRAS [4], collected from real access and edge devices; and CICIDS2017_Monday [14], originating from a well-known synthetic dataset. [5] already provide the distributions for its datasets, while for [4, 14] we used the tool developed by [5] to generate the distributions from raw traces.

These datasets offer contrasting flow length distributions, as AGH contains large flows that carry the majority of packets, BRAS lacks large flows altogether, and CICIDS2017_Monday serves as an intermediate distribution between them, though with a higher proportion of single-packet flows. The goal of using distinct models is to evaluate how RSS, multi-threading, and the application handle diverse flow distributions. To generate traffic as faithfully as possible to these distributions (while adhering to the limitations in our TRex scripts), we generated four million unique flows for each model, continuously sending AGH traffic at 35Gbps, BRAS traffic at 10Gbps, and CICIDS2017_Monday at 30Gbps.

Finally, for the experiments involving Suricata, we ensured that its flow decoding and processing stage had sufficient memory to avoid saturation (i.e., *memcap.pressure* remains low). Moreover, all tools were configured according to their optimal deployment parameters (e.g., disabling checksum and enabling busy-polling for AF_XDP), and we provided a comparable amount of memory for each strategy (more or less 16 MB per thread, as this is the hardcoded value for AF_XDP). Finally, we use the publicly-available emerging threats rulesets[4].

[4] https://rules.emergingthreats.net/

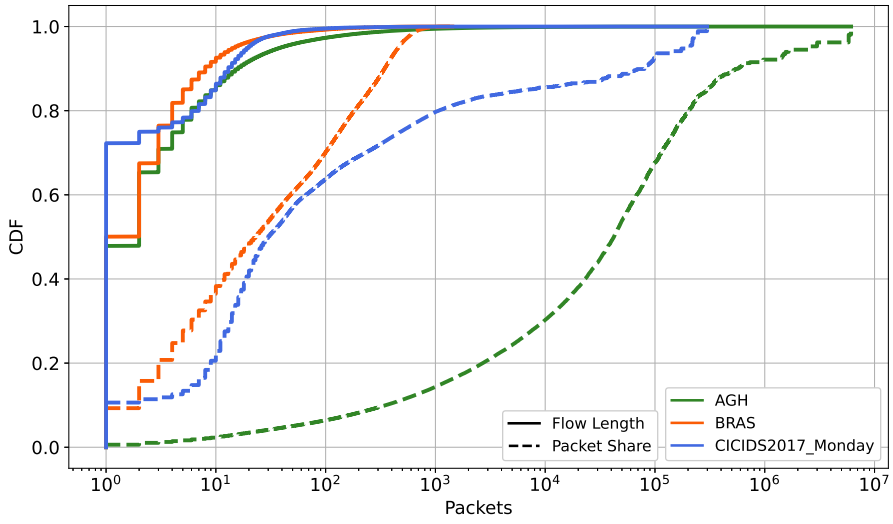


Fig. 4: Flow length distribution and packet share per flow length for the AGH, BRAS, CICIDS2017_Monday traffic models.

5 Results

This section presents the results of the experiments described before.

5.1 Single thread performance

The single-thread experiments were performed to evaluate the raw packet processing performance of a single thread. Figure 5 presents the results across the four evaluated throughput rates, where the y-axis indicates the percentage of packets received by each application relative to the total number of transmitted packets.

As illustrated, DPDK shows the best raw packet processing performance, processing all incoming packets except when the packet size is 64 bytes and throughput is equal to or above 25Gbps. This outcome is expected, as DPDK provides a direct interface to the NIC's DMA while utilizing *hugepages* to ensure high memory availability. Second to DPDK is AF_XDPb, where the suffix "b" indicates that this configuration employs busy-polling. This slight modification significantly increases AF_XDP's performance, as packets are constantly being retrieved from the NIC rather than waiting for interrupts. Although both AF_XDP and AF_XDPb use the `XDP_ZERO_COPY` option (where the UMEM is DMA-accessible), the packet still traverses the XDP program and the NAPI scheduler, which explains why neither reaches the same performance level as DPDK. Finally, both AF_PACKET and vanilla PF_RING demonstrate similar and limited performance, as both tools share virtually the same processing logic per thread and still require packets to pass through the kernel's network stack.

The results of Figure 5 shows only the results of the best configuration for each framework. We noticed that increasing the memory capacity (i.e., the *mempool* size for DPDK, the number of packets in the *mmap* buffer, or the *UMEM* size) across all packet processing frameworks did not yield any performance improvements. This behavior is expected because the benchmark application itself does not perform expensive computations. However, for more complex applications, such as intrusion detection with Suricata, memory size should impact overall performance.

5.2 Multiple threads performance

The second set of experiments evaluates how effectively each tool scales with multiple threads and RSS. Figure 6 presents the results for the three flow profiles described in Section 4.2. As anticipated from the single-thread benchmark results, DPDK achieves the highest performance, followed by AF_XDP with busy-polling, then standard AF_XDP, AF_PACKET, and finally, PF_RING. Although DPDK is the only tool that handles all packets using a single thread, AF_XDP with busy-polling also processes all packets when allocated four or more threads.

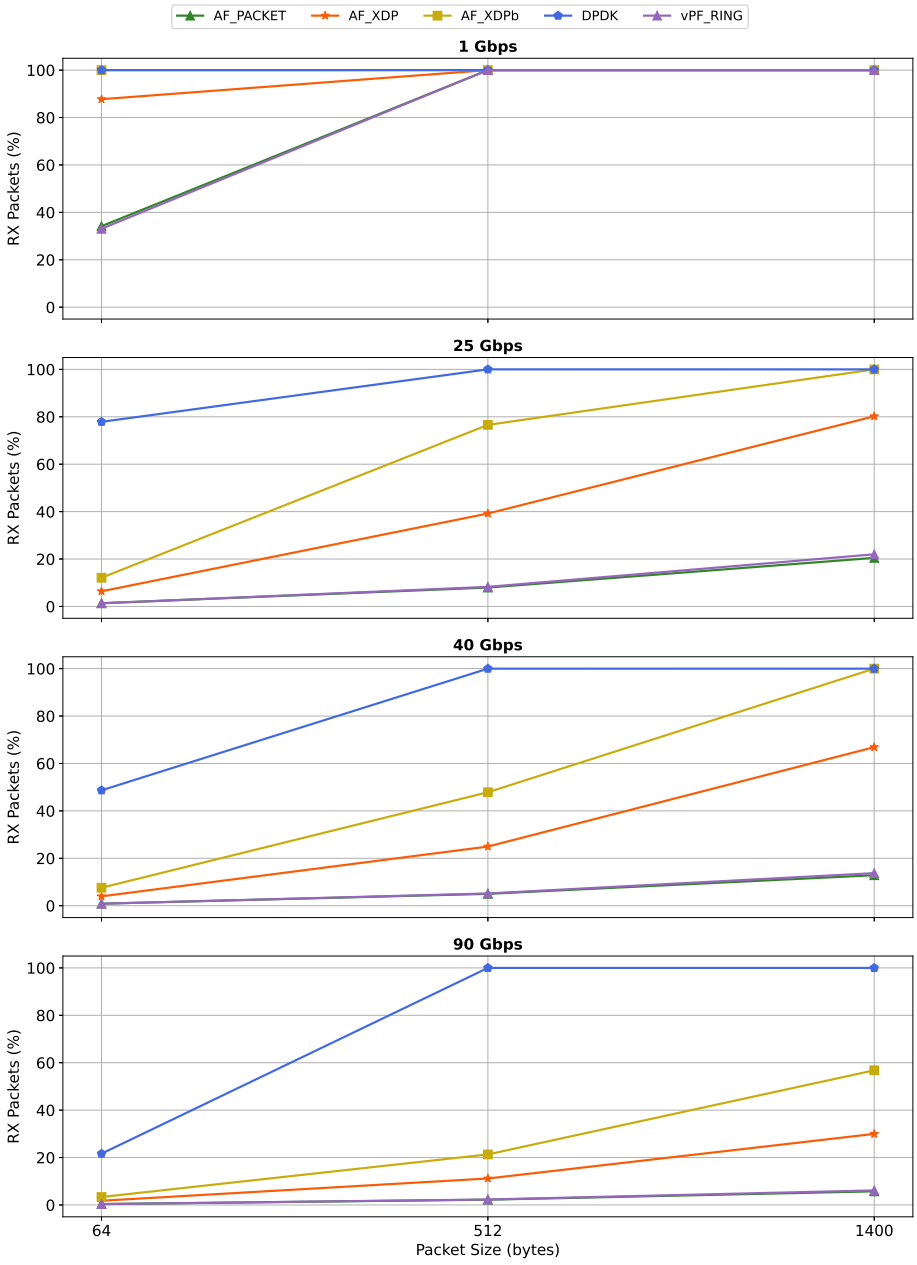


Fig. 5: Single-threaded experiment with varying speeds and packet sizes.

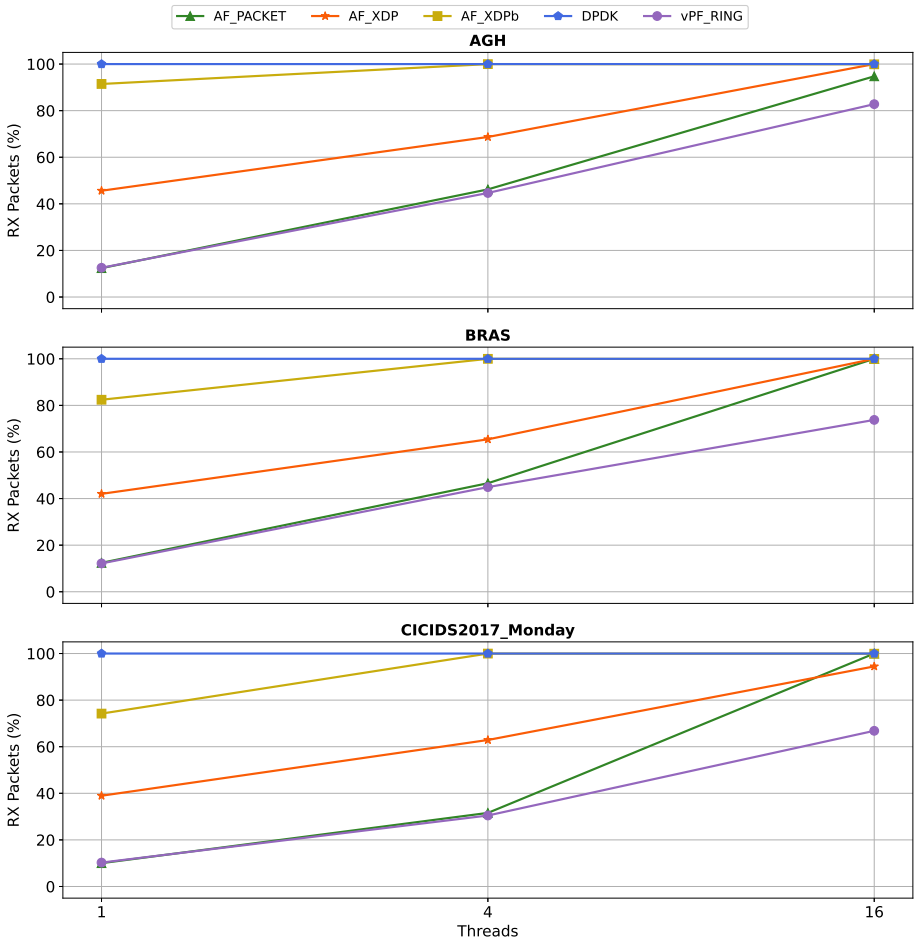


Fig. 6: Multi-threaded experiment with different flow length distribution profiles.

Surprisingly, AF_PACKET manages to process all packets under both the BRAS and CICIDS2017_Monday profiles when using 16 threads. This demonstrates that its multi-threaded architecture can fully exploit RSS when configured with the appropriate socket fanout mode. Rather than using `PACKET_FANOUT_HASH` (which leads to unnecessary inter-core communication), applying `PACKET_FANOUT_CPU` ensures packets remain on the CPU cores where they were originally received. This mechanism also explains why PF_RING fails to scale comparably to AF_PACKET, as it lacks this balancing mode that considers the initial RSS mapping.

Despite differences in link speeds and flow length distributions, all scenarios yield similar performance trends, with AGH leading to slightly superior results across all tools, whereas CICIDS2017_Monday results in the lowest performance.

This small discrepancy can be explained by the nature of each profile, as CICIDS2017_Monday contains numerous short flows that generally produce a less stable per-flow workload, while AGH and its elephant flows provide a more consistent workload, where most packet processing costs are concentrated within a few established connections and contexts. This also explains why BRAS causes virtually the same costs as the other profiles, despite operating at only 10Gbps.

5.3 Suricata performance

The final set of experiments uses Suricata to assess how the raw packet processing capabilities of each tool translates to actual application performance. We used the same set of flow distributions from Section 5.2. Figure 7 shows the percentage of packets inspected by Suricata (lower green bar), the number of packets received by Suricata through the packet processing tool but dropped before inspection (middle yellow bar), and the number of packets that the tool failed to receive altogether (upper orange bar). For these experiments, we performed five independent runs to account for run-specific anomalies in Suricata and calculated the mean and standard deviation (SD)

Figure 7 contains many interesting results. First, DPDK always manages to deliver all packets to Suricata (similar to the previous experiments), but that does not translate to Suricata actually inspecting all packets. The exact cause for this behavior is not entirely clear. Although we posted an inquiry in Suricata's community forum[5], we did not receive any definitive follow-up regarding what might be causing this discrepancy. The probable cause lies in the processing steps required to convert a raw frame into a structured packet format within the application context, as well as the management of DPDK's internal structures (*mempools* and rings).

Second, although AF_XDP fails to deliver all packets to Suricata (which differs from the results in Figure 6), all packets that are received by Suricata are actually inspected. It is the only tool that achieves near-complete inspection of its received packets. This is particularly surprising given that DPDK delivers all packets to Suricata, yet consistently results in a lower percentage of inspected packets. The probable cause is a more efficient raw-packet-to-structured-data conversion and better internal management of AF_XDP structures. As for the significant reduction in received packets compared to the minimal application benchmark in Section 5.2, the added computational overhead of decoding packets, managing flows and streams, as well as inspecting payload content explains this difference.

Third, the tools with the highest kernel overhead, AF_PACKET and PF_RING, exhibited the lowest total packet reception rates, with AF_PACKET achieving superior performance compared to PF_RING across all flow profiles. For these two frameworks (and only them), substantial packet loss occurred at two distinct stages: first, when the kernel fails to deliver the packet to the socket, and second, when the socket itself fails to pass the packet to Suricata. This

[5] https://forum.suricata.io/t/dpdk-mode-is-dropping-too-many-packets/6149

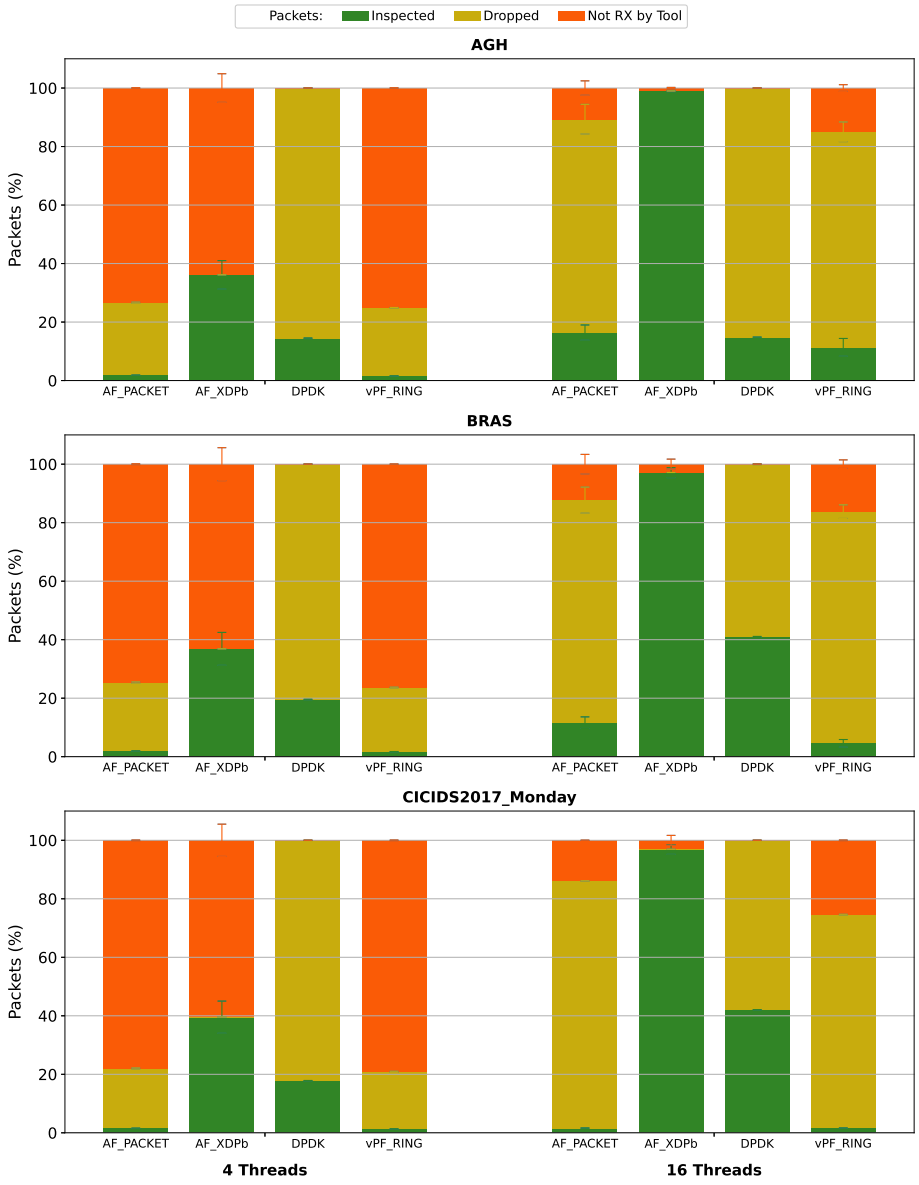


Fig. 7: Suricata experiments with different threads and flow length distribution profiles.

second application-related loss stage occurred exclusively during the Suricata experiments, as the baseline counting application reported zero drops at the application layer.

Finally, we noticed that increasing the memory allocation for DPDK (*mempool* size and descriptors), AF_PACKET (*ring_size*), and PF_RING (*min_num_slots*) did not produce any noticeable difference in the number of packets inspected by Suricata. For example, in the AGH scenario with AF_PACKET, increasing memory to 1024 MB per thread kept the number of received packets within the same range, while failing to reduce packet loss to an acceptable rate (less than 1%). With four threads, the proportion of dropped packets was reduced by roughly 30 percentage points (though still resulting in 60% of packets dropped), while with 16 threads, the reduction was only 12 percentage points, causing 72% of all packets to be dropped. Moreover, further tests with AF_PACKET while adjusting the NAPI budget (i.e., *net.core.netdev_budget*) to 600 and 2000 packets did not yield substantial improvements. Taken together, these findings suggest that memory capacity is not the main bottleneck, but rather the processing speed at which raw packets are converted into a format suitable for decoding and subsequent inspection by Suricata.

6 Conclusion

Packet processing frameworks, such as AF_PACKET and DPDK, represent the final step before packets reach their destination application. With ever-increasing network speeds, as exemplified by the latest 400 and 800Gbps NICs, such systems must operate efficiently to handle these high packet volumes. In this work, we assessed four well-known tools under diverse operating conditions and different target applications. Our results demonstrate that the unmatched raw packet processing performance of DPDK does not translate to superior application performance. Instead, AF_XDP showed the best overall performance when integrated with Suricata, inspecting more than 95% of all transmitted packets with 16 threads across all scenarios, whereas Suricata using any of the other frameworks inspected at most 45%.

References

1. Broadcom: Ethernet Network Adapters (2026), https://www.broadcom.com/products/ethernet-connectivity/network-adapters
2. Corporation, N.: NVIDIA Ethernet SuperNICs (2026), https://www.nvidia.com/en-us/networking/products/ethernet/supernic/
3. Høiland-Jørgensen, T., Brouer, J.D., Borkmann, D., Fastabend, J., Herbert, T., Ahern, D., Miller, D.: The express data path: Fast programmable packet processing in the operating system kernel. In: Proceedings of the 14th international conference on emerging networking experiments and technologies. pp. 54–66 (2018)
4. Jiang, W., Zhang, B., Zhu, Q., Liao, C., Wang, W.: A real network environment dataset for traffic analysis. Scientific Data **12**(1), 756 (2025)
5. Jurkiewicz, P., Rzym, G., Boryło, P.: Flow length and size distributions in campus internet traffic. Computer Communications **167**, 15–30 (2021)
6. Karlsson, M., Töpel, B.: The path to dpdk speeds for af xdp. In: Linux Plumbers Conference. vol. 37, p. 38 (2018)

7. Larin, D., Get'man, A.I.: Tools for capturing and processing high-speed network traffic. Programming and Computer Software **48**(8), 756–769 (2022)
8. Linux: packet(7) - Linux manual page — man7.org. https://man7.org/linux/man-pages/man7/packet.7.html (2026), [Accessed 23-10-2026]
9. Mostafa, J., Chilingaryan, S., Kopmann, A.: Are kernel drivers ready for accelerated packet processing using af_xdp? In: 2023 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN). pp. 117–122. IEEE (2023)
10. ntop: PF_RING – ntop — ntop.org. https://www.ntop.org/products/packet-capture/pf_ring/ (2026), [Accessed 23-10-2026]
11. Open Information Security Foundation (OISF): Suricata. https://suricata.io/ (2026), [Accessed 23-10-2026]
12. Parola, F., Procopio, R., Querio, R., Risso, F.: Comparing user space and in-kernel packet processing for edge data centers. ACM SIGCOMM Computer Communication Review **53**(1), 14–29 (2023)
13. Projects, T.L.F.: DPDK – The open source data plane development kit accelerating network performance — dpdk.org. https://www.dpdk.org/ (2026), [Accessed 23-10-2026]
14. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., et al.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSp **1**(2018), 108–116 (2018)
15. Šišmiš, L.: Optimization of the suricata ids/ips (2021)
16. Töpel, B., Karlsson, M., Duyck, A., Starovoitov, A., Borkmann, D., Dangaard Brouer, J., Fastabend, J., Corbet, J., S. Tsirkin, M., Z. Zhang, Q., de Bruijn, W.: AF_XDP — The Linux Kernel documentation — docs.kernel.org. https://docs.kernel.org/networking/af_xdp.html (2026), [Accessed 23-10-2026]
17. Valia, S., Rajput, N., Gairola, A., Rathi, V.: Comparing upf dataplane i/o modes in a cloud-native 5g core: Af_packet, af_xdp, cndp, and dpdk on sd-core bess-upf. arXiv preprint arXiv:2608.14589 (2026)